
Report of Final Project

1 Introduction

Grokking is a dramatic phenomenon in machine learning that generalization happens far after overfitting. In this project, we will explore the grokking phenomenon in the learning of modular addition: $(x, y) \mapsto (x + y) \bmod p$, $x, y \in \mathbb{Z}_p$, where p is a prime number. We will reproduce the experiments in [3] for modular addition, and then generalize our discussion to different models including MLP and LSTM, and also to the impact of different optimizers and regularization methods.

Furthermore, we will give an explanation of the grokking phenomenon. We find there are two aspects that explain this phenomenon well. One aspect is the decay of l_2 norm of model parameters, and the other is the process of learning the commutative law of modular addition.

The rest of the report is organized as follows. In Section 2, we will introduce the problem setting, including choices of parameters, and how the inputs are processed before entering the model. In Section 3, we will establish the results of our experiments corresponding with Subtasks 1, 2, and 3. In Section 4, we will give our explanations of the grokking phenomenon, which corresponds with Subtask 4.

2 Problem Setting

Before showing our main results, we make a brief elaboration of our problem settings here. The parameter settings (mostly similar to the experiments in [3]) are as follows. We choose $p = 97$ for all tasks. The original inputs are in the form of $\{(x, \text{op}, y, \text{eq}) : x, y = 0, 1, \dots, p - 1\}$, where op and eq represent for $+$ and $=$ in the equations, and are given value of op = p and eq = $p + 1$, respectively. To make the information of “integers” not known to the models, each token is transformed into a 128-dimensional embedding vector by an embedding mapping

$$a \mapsto E_a \in \mathbb{R}^{128}, a \in \{0, 1, \dots, p + 1\},$$

so the inputs after embedding are all matrices in $\mathbb{R}^{4 \times 128}$. The outputs are of $p + 2$ dimensions, corresponding with $0, 1, \dots, p + 1$. As for the optimizer, we also choose AdamW optimizer with learning rate 10^{-3} , weight decay 1, $\beta_1 = 0.9$, $\beta_2 = 0.98$, also linear learning rate warmup over the first 10 updates starting with 0.1 times the learning rate. Since we treat this as a classification task, the cross-entropy loss function is used to evaluate the model.

3 Experiments

3.1 Grokking in transformer

This section is about reproducing the grokking phenomenon of transform shown in [3] but for modular addition. To be specific, we will use a transformer model with 2 layers, embedding dimension 128, and 4 attention heads. Here we take a batchsize of 256 and a maximal optimization budget of 5×10^5 gradient updates.

We take two different ways to formulate the dataset, one selecting all $x, y \in \mathbb{Z}_p$ (referred to as full dataset) and the other selecting only those satisfying $x \geq y$ (referred to as half dataset), then dividing

the data into the training dataset and validation dataset according to the parameter of training fraction. For the code writing part of this section, we’ve referred to the code of [2]. The results for both will be displayed in this section.

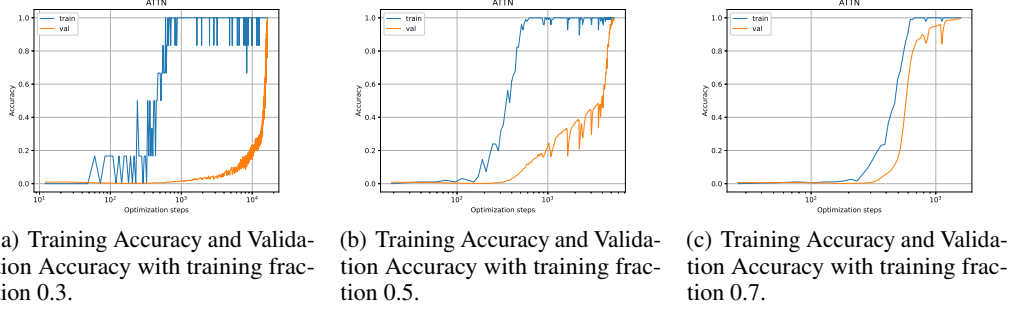


Figure 1: Training Accuracy and Validation Accuracy with different Training Fraction for Full Dataset

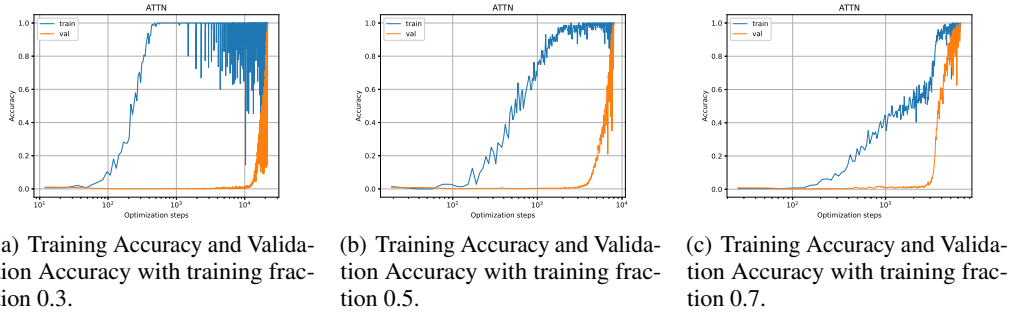


Figure 2: Training Accuracy and Validation Accuracy with different Training Fraction for Half Dataset

As Figure 1 has shown, one can observe the grokking phenomena in the training process of modular addition with a relatively smaller training fraction for full dataset. But there are still some essential differences from the original figure in [3], greatly influenced by the commutativity of the data, which would be further explored in Section 4. Figure 2 for half dataset show the grokking curve more obviously due to similar reasons.

3.2 Grokking in other models

To see whether grokking only happens in transformer, we also test the same problem on MLP and LSTM. Specifically, we will use a 3-layer MLP with architecture $512 \rightarrow 128 \rightarrow 64 \rightarrow 64 \rightarrow 99$, and a LSTM with 2 layers. The training data fraction α is set as 0.5. Weight decay is set as 0.05 here to see more obvious grokking. The results are shown in Figure 3.

We can see very significant grokking phenomena from Figure 3. However, the differences between these results and Figure 1(b) imply that the two models do not have good abilities to learn commutative law before generalization. This point will be further explained in Section 4.2.

There is another notable point. When changing the structure of models, the grokking phenomenon still exists, which shows grokking is not caused by specific structures of models but is a general phenomenon. Relevant results are shown in Figure 4, where an MLP with architecture $512 \rightarrow 128 \rightarrow 64 \rightarrow 99$ is used.

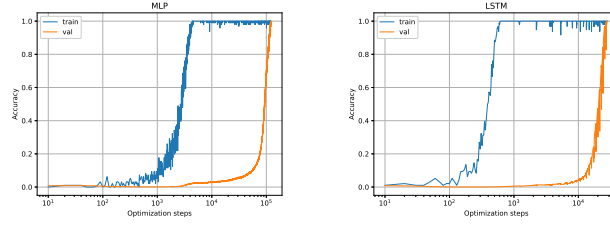


Figure 3: Training accuracy and validation accuracy of different models. **Left:** MLP with 3 layers. **Right:** LSTM.

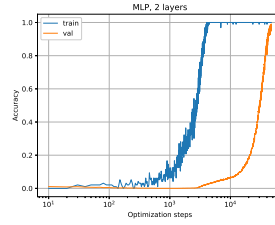


Figure 4: Training accuracy and validation accuracy of MLP with 2 layers.

3.3 Impact of optimizers and regularization methods

Next we will show the impact of different optimizers and regularization methods. Here we present the data efficiency curves on modular addition for the following varieties of AdamW: different weight decays, residual dropout, and smaller learning rate. We have also tested the efficiency of SGD accelerated by Nesterov momentum. The results are shown in Figure 5. Experiments are only done on one random seed because of the limit of computility, but different optimization methods already show significant differences in data efficiency.

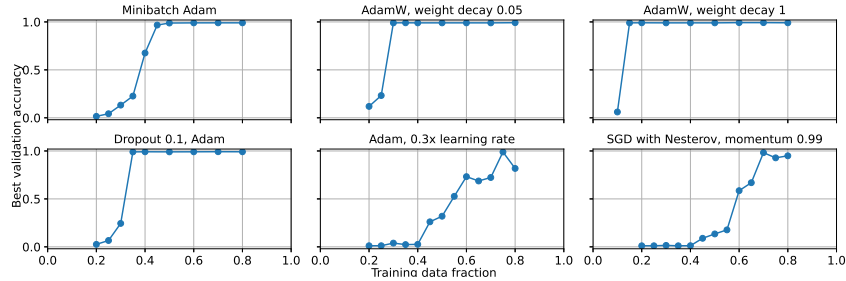


Figure 5: Data efficiency curves of different optimization methods. The x -axis is the training data fraction α , and the y -axis is the best validation accuracy achieved within an optimization budget of 10^5 steps.

Figure 5 shows results similar to [3, Section 3.3]. We can see that weight decay for Adam has a great effect on improving generalizations, especially for small α . Also, adding a residual dropout to the model also increases the data efficiency, but is not as effective as weight decay. Taking a smaller learning rate diminishes the ability of generalization, which shows the learning rate we take is appropriate. By implementing Nesterov momentum for accelerating, SGD also shows similar performance to the worst case of Adam (small learning rate with no weight decay). Therefore, AdamW is an efficient optimizer for this problem.

4 Explanation of Grokking

In this section, we'll give a brief explanation of the grokking phenomenon. The explanation will mainly focus on two aspects: the decay of l_2 norm and the commutativity of modular addition for this special case.

4.1 Decay of l_2 norm

It is actually quite intuitive to see that the l_2 norm of the model parameters plays an important role in the forming of grokking phenomenon. The article [4] has also shown analytical results lower-bounding test accuracy with the value of weight norm. Plotting the weight-norm curve (shrunk by the size of the dimension of the model, which is 128, to have its scale matching that of the accuracy curve) against the optimization steps and comparing it with the accuracy curves, one can see clearly that long after the convergence of training accuracy to one, the l_2 weight norm is still decreasing with a growing decrease rate. And when the weight norm is low enough, the model begins to generalize with the validation accuracy increasing rapidly from random guess level to one. This process might lead to the weight norm increasing again (In some cases it won't increase as in Figure 6), as the generalization process would have the loss decreasing fast enough again. Then after the validation accuracy converges, the weight norm would decrease again. Such relations can be seen in Figure 6.

To control the l_2 norm of the model, it is intuitive to control the weight decay factor of the optimizer. Proper weight decay factor would lead to grokking. Too large weight decay, with its great impact on the loss function, would lead to non-convergence of even the training accuracy; while too small weight decay might perform badly in generalization, showing no trial of generalization in the limited steps. Therefore, choosing smaller weight decay factors would more likely lead to the grokking phenomenon in practice as long as generalization can happen.

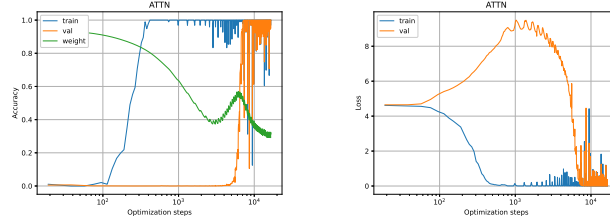


Figure 6: Weight Norm, Accuracy and Loss of a grokking example, with Definition 1 commutative fraction 0 and weight decay 0.5. **Left:** Accuracy and weight norm. **Right:** Loss.

4.2 Commutativity of modular addition

For the discussion of this section, we consider only nearly equal-size training dataset and validation dataset, which means the training data fraction is set to 0.5.

It can be seen that grokking isn't obvious for full dataset, nor does the weight decay trick given in the previous section work well, mainly due to the slow increasing period of validation accuracy shortly after training accuracy begins to rise (Figure 1). Inspired by the idea of half dataset used in [1], we find the grokking phenomenon more obvious on half dataset as well as that when learning modular division. The structural difference lies in the commutativity of the operation to be learned (though modular addition is commutative, half dataset does not make use of it). To push this idea forward, we'll introduce the definition of commutative fractions.

Definition 1 (Commutative Fraction). *Let A denote the set defined by $A = \{(x, y) | (x, y) \in T \text{ and } (y, x) \in V\}$, where T denotes the training dataset and V denotes the validation dataset. Then commutative fraction is defined by $\frac{|A|}{|T|}$.*

In this setting, full dataset has a commutative fraction of nearly 0.5 due to the law of large numbers, and half dataset is essentially closer to that with a commutative fraction of 0.

In Figure 7, it can be seen that with a smaller commutative fraction (from 0.1 to 0.3), the learning process obviously decouples the learning of commutative law and the generalization process. With a smaller commutative fraction, there are more pairs of (x, y) and (y, x) 's in the training dataset, enabling it to learn the commutative law of modular addition soon after the training accuracy converges. Then in the long period before the generalization, the validation accuracy would remain close to the commutative fraction, as the model could only answer correctly those (y, x) 's in the validation dataset whose commuted version (x, y) is in the training dataset. After this period the generalization takes place and validation accuracy converges.

This also explains why half dataset (Figure 2) or dataset with commutative fraction 0 (Figure 6) would exhibit the grokking phenomenon, as the learning of commutative law won't influence the validation accuracy at all.

Figure 7 shows that learning commutative law and generalization are two rather separate processes, while grokking comes from the latter. With full dataset, where the commutative fraction is nearly 0.5, the two processes coincide, making the grokking phenomenon not so obvious. With the commutative fraction even larger (0.7 to 0.9), with fewer pairs of data in the training dataset, the effect of learning of commutative law gets worse and its starting points are pushed backward, exhibiting again the phenomenon of grokking.

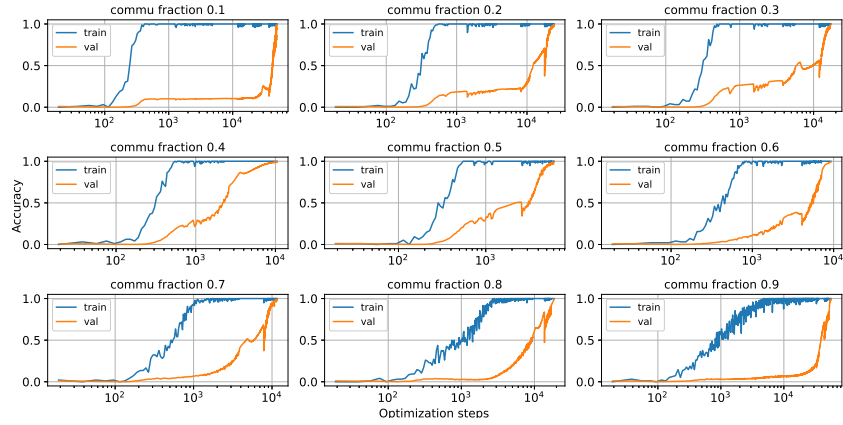


Figure 7: Accuracy of the model with commutative fraction from 0.1 to 0.9 and weight decay 0.05.

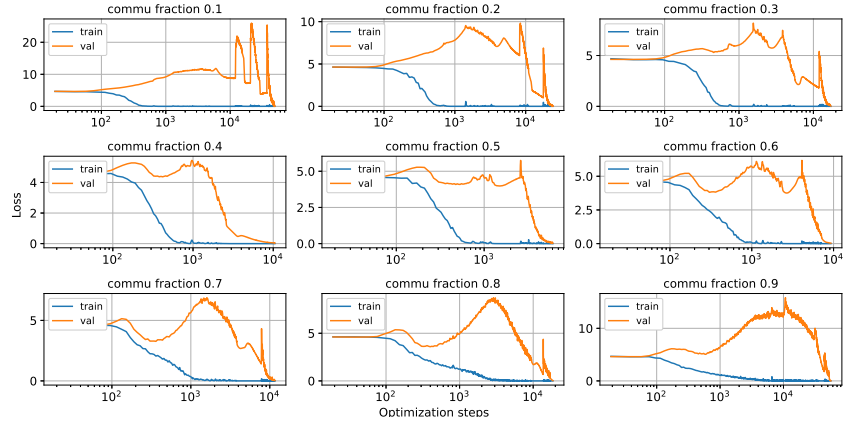


Figure 8: Loss of the model with commutative fraction from 0.1 to 0.9 and weight decay 0.05.

References

- [1] Z. LIU, O. KITOUNI, N. S. NOLTE, E. MICHAUD, M. TEGMARK, AND M. WILLIAMS, *Towards understanding grokking: An effective theory of representation learning*, Advances in Neural Information Processing Systems, 35 (2022), pp. 34651–34663.

- [2] D. MAMAY, <https://github.com/danielmamay/grokking>, 2023.
- [3] A. POWER, Y. BURDA, H. EDWARDS, I. BABUSCHKIN, AND V. MISRA, *Grokking: Generalization beyond overfitting on small algorithmic datasets*, arXiv preprint arXiv:2201.02177, (2022).
- [4] Z. TAN AND W. HUANG, *Understanding grokking through a robustness viewpoint*, arXiv preprint arXiv:2311.06597, (2023).